

Database Relationships

Section Review

Database Relationships

- A database relationship is a connection between two tables.
- Relationships connect related ideas within our business domain.
- In our Rent a Room API, a booking needs to know which room and user it belongs to.
- Relationships in relational/table databases use **foreign keys**.
A foreign key is a column whose values reference the primary keys of another table.

The Bookings Table

- The **room_id** column on the **bookings** table became the foreign key pointing to a room.
- The **user_id** column on the **bookings** table became the foreign key pointing to a user.
- The **bookings** table can be described as a join table because its primary purpose is to connect other tables' records together.

The Relationship Function I

- A **Relationship** attribute on a SQLAlchemyModel can navigate directly to one or more related model records.
- On the Room model, we defined a **bookings** attribute because a room can have many bookings.
- On the Booking model, we defined a **room** attribute because a booking belongs to one room.

The Relationship Function II

- The **back_populates** parameter updates the complementary model when one model's state/association updates.
- Provide a string arguments that refers to the name of the complementary attribute on the *other* model.

Circular Imports

- We can't import each model in the other model's file because that would create a circular import.
- We used the **TYPE_CHECKING** constant to ensure that only the type checker imported the models.
- The runtime code/FastAPI does not run the import code.
- We referred to the models with strings. The technical term for this is a "forward reference".

Creating a Booking

- The new **POST /bookings** endpoint accepts a **room_id** and **user_id** in the payload.
- The client also provides **check_in** and **check_out** times.
- The datetime values arrive as strings but Pydantic converts them into Python datetime objects and the database stores them in datetime columns.

Eager Loading vs. Lazy Loading

- Synchronous database sessions use **lazy loading**. The ORM fetches an associated database record when code accesses a relationship attribute.
- **Eager loading** performs the fetch *earlier before* the supplementary data is needed.
- Async sessions require eager loading of the complementary records *first*. We can then reference their associations in the code.

selectinload and joinedload

- The **selectinload** function makes two requests to the DB, one for the initial row and the second for associated records. It is ideal for a one-to-many relationship (like a room and all of its bookings).
- The **joinedload** function makes one request to the DB, joining together a record with associated records. It is ideal for a one-to-one relationship (like a booking and its user).

Dealing with Deletions

- When a primary record is deleted, the **CASCADE DELETE** setting deletes records that reference the primary record through a foreign key. For example, a room deletion can delete all associated bookings referring to that room.
- The **SET NULL** setting sets a row's foreign key column value to **NULL**, the database representation of nothingness. The supplementary record remains but now has no link to the (deleted) primary record.

Types of Joins

- An **inner join** merges two tables together based on matching values across columns. Records that have no matches are excluded.
- An **outer join** merges two tables together, matching records together when possible but still retaining other records from one or both tables when there is no match.

ORM Querying I

- The **select** function and the model argument determines what table's data is returned back to the ORM.
- The **select** function accepts multiple arguments. You can return joined rows from multiple tables or the evaluation of additional expressions.
- When joining another table with **join** or **outerjoin**, we can use its columns for filtering/grouping but the columns' values won't be included in the data that the DB returns.

ORM Querying II

- The **exists** function validates that an associated record exists in another table without fetching its data.
- An ORM supports SQL functions like **COUNT** and **LOWER**. Import the **func** object from **sqlmodel**.
- The **group_by** method organizes records into groups/buckets based on a column's value.